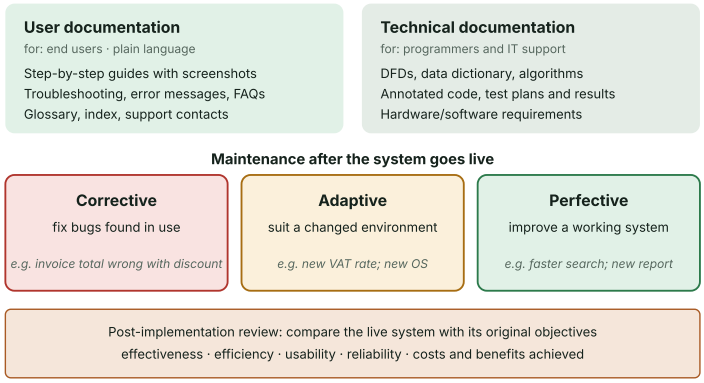
Documentation, maintenance and evaluation

AP09 FACT FILE

Specification: students should be able to

- Explain why documentation is produced at each stage of system development
- Describe user documentation and technical documentation and explain how each is used
- Describe the purpose of system maintenance and the reasons it is needed
- Describe and evaluate corrective, adaptive and perfective maintenance
- Describe the purpose and content of a post-implementation review

Key words: documentation, user documentation, technical documentation, user guide, troubleshooting, FAQ, system maintenance, corrective maintenance, software patch, adaptive maintenance, perfective maintenance, maintenance contract, annotated code, post-implementation review, evaluation criteria



User and technical documentation compared, the three types of maintenance with examples, and the post-implementation review

Overview: documenting and maintaining a system

Once a system is live, two activities keep it useful: good documentation and ongoing maintenance. Documentation is produced at every stage of development, not just at the end. Each stage's documents provide the starting point for the next, such as the requirements specification guiding the designers and the design document guiding the programmers; they record the work done; they support maintenance by keeping track of what was built and what has since changed; and they allow new staff to take over if members of the development team leave. For the finished system there are two main types. User documentation helps the people who use the system day to day, while technical documentation helps the administrators and programmers who look after it. Maintenance is any change made to a system during its working life. However carefully a system is designed and tested, it will need changing: to fix errors, to adapt to new circumstances, or to make improvements. These are known as corrective, adaptive and perfective maintenance. Finally, a post-implementation review evaluates whether the system has achieved the objectives set at the start of the project.

User documentation

User documentation is written for the end users of a system, so it uses plain language, avoids unnecessary jargon and is organised around the tasks people want to carry out. It typically begins with an overview of what the system does, followed by the hardware and software needed to run it and any installation instructions. The core is a user guide giving step-by-step instructions for common tasks, usually illustrated with screenshots. A troubleshooting section explains what to do when things go wrong, including the meaning of error messages, and a frequently asked questions section answers common queries quickly. Additional training material, such as tutorials or practice exercises, may be included, together with routine procedures such as how and when to back up data. User documentation can be supplied as a printed manual, an online help system, context-sensitive help that appears for the part of the screen being used, or short video tutorials. Good user documentation reduces calls to the help desk, speeds up training and helps users become confident and independent with the new system.

Technical documentation

Technical documentation is written for the system administrator, the maintenance staff and any users with strong technical knowledge. Its purpose is to let someone who did not build the system understand exactly how it works, so that it can be maintained, repaired and extended. It contains the system specification and user requirements; data models such as data flow diagrams, entity relationship diagrams and the data dictionary; the input and output designs, report specifications and query designs; the program code listings, which should be annotated with comments and use meaningful identifiers; the test plans together with the test data used and the results obtained; and detailed descriptions of the hardware and software configuration, such as server, network and database settings. Without it, every change would require programmers to work out the system from the code alone, which is slow, expensive and likely to introduce new errors. Technical documentation must itself be kept up to date whenever maintenance is carried out, otherwise it becomes misleading. It is especially important because the original developers may have left by the time changes are needed.

Why maintenance is needed

System maintenance covers every change made to a system after it goes live, for as long as it remains in use. Just as a car needs servicing, a computer system needs attention for as long as it is used, however rigorously it was designed and tested. Several things trigger maintenance. A software error may be uncovered during normal use that testing did not find. Some part of the organisation's business processes may change, so the system must be modified to match. A security vulnerability may be discovered that must be closed before it is exploited. Someone may identify a way of improving the system. The operating infrastructure, such as hardware or networks, may be upgraded, and the system can be changed to take advantage of it. These reasons fall into three broad categories: corrective, adaptive and perfective maintenance. Because maintenance continues for the whole life of a system, which may be many years, its total cost frequently exceeds the cost of the original development. Modular design, annotated code and accurate technical documentation all reduce that cost by making changes quicker and safer to carry out.

Corrective, adaptive and perfective maintenance

Corrective maintenance fixes flaws or weaknesses found once the system is in use, such as calculations that give the wrong result or a security hole that leaves the system open to attack. It is most urgent when the system is insecure or cannot carry out its basic tasks. Fixes are often supplied as software patches, small updates that are downloaded and frequently installed automatically. The original developers usually carry out the work, free of charge if the problem was caused by programmer error, or otherwise under a rolling maintenance contract, and the amount needed should fall over time as errors are removed. Adaptive maintenance changes the system because circumstances have changed, either externally, such as a new tax rate or new legislation, or internally, such as a revised discount policy. The system was correct when built but can no longer do its job without change. The client normally pays, since new functionality is being added, and a large amount soon after delivery may suggest the requirements were poorly analysed. Perfective maintenance improves a system that already works, for example by redesigning forms, improving help, adding shortcuts or archiving data to speed up searches.

Post-implementation review

A post-implementation review is a formal evaluation carried out once the system has been in use long enough for problems and benefits to show, often several months after go-live. It judges the system against the objectives and requirements agreed during analysis rather than against personal opinion. Evidence is gathered from user questionnaires and interviews, observation of the system in use, performance statistics such as response and processing times, help-desk and error logs, and financial records that show whether the costs and savings predicted in the feasibility study were achieved. The review considers effectiveness, meaning whether the system meets its requirements; efficiency, meaning whether it is fast and economical to run; usability, meaning whether staff find it easy to use; reliability, meaning how often it fails or loses data; and whether the expected benefits have been delivered. Its findings identify outstanding problems and recommend corrective, adaptive or perfective maintenance, and it records lessons learned so that future projects in the organisation can be managed better. Evaluation is different from testing: testing happens before handover, whereas evaluation measures success after implementation.

T Key terms from the mark schemes

User documentation
Non-technical documentation for end users, such as an overview, installation instructions, user guide, troubleshooting, FAQs and backup procedures.

Technical documentation
Documentation for administrators and maintenance staff, including specifications, DFDs and data models, I/O and report designs, annotated code, test plans and results, and hardware/software configuration.
`Credited: system documentation`

System maintenance
Any change applied to a system during its lifespan after it has been implemented.
`Credited: maintenance`

Corrective maintenance
Changes made to fix flaws, errors or weaknesses, including security flaws, discovered after the system is in use.

Adaptive maintenance
Changes made so that the system can still carry out its tasks after a change in circumstances, internal or external, such as new legislation, a new tax rate or a new business policy.

Perfective maintenance
Changes that improve the performance or usability of a system that already works, after initial delivery.

Software patch
A small update released to correct, change or enhance a program or its data, often downloaded and installed automatically.
`Credited: patch / update`

Annotated code
Program code that includes comments explaining its purpose and logic, so that others can understand and maintain it.
`Credited: commented code`

Post-implementation review
A formal evaluation, after a system has been in use for some time, of how well it meets its original objectives and requirements.
`Credited: evaluation`

C C# example

`Annotated code is technical documentation`

```
// CalculateFee: returns the monthly fee for a gym member
// Parameters: age in years, isStudent (true if full-time student)
// Rules agreed with client (requirements spec, section 4.2):
//   under 18 = £15, student = £20, everyone else = £32
decimal CalculateFee(int age, bool isStudent)
{
    if (age < 18) return 15m;
    if (isStudent) return 20m;
    return 32m;
}
Console.WriteLine(CalculateFee(16, false)); // 15
Console.WriteLine(CalculateFee(21, true));  // 20
```

Exam focus

Documentation questions often ask you to list components, usually one mark each. Give specific, different items, such as installation instructions, a troubleshooting section and FAQs for user documentation, or annotated code listings, data flow diagrams and test plans with results for technical documentation. Saying that user documentation ‘tells people how to use the system’ is too vague to earn more than one mark. If asked why documentation is produced at each stage, give distinct reasons: a starting point for the next stage, a record of work done, help with maintenance, and continuity when team members change. Maintenance questions frequently describe a series of changes and ask you to classify each. Ask yourself whether the system was wrong when delivered (corrective), whether something around the system has changed (adaptive), or whether a working system is being made better (perfective). A change caused by new law is adaptive, not corrective, because the system met its requirements when built. When defining each type, add a short example, as definitions with examples are commonly credited together. Do not confuse the post-implementation review with testing.

? Check yourself

1. A software house in Banbridge is building a records system for a veterinary practice. Explain why the team should write documentation throughout the project rather than only at the end. [4]
2. State three components of user documentation for an online appointment system used by patients of a Ballymena GP surgery. [3]
3. A Dungannon engineering firm's in-house IT technician has to maintain a stock system written by a contractor who has since left. State three components of technical documentation that the technician would need. [3]

Answers: 1. Any four from: each stage's documents give the next stage something to work from, e.g. programmers code from the design document (1) / they record what was done and decided at each stage (1) / they help later maintenance by showing how the records system was built and what has changed (1) / new staff can pick up the work if team members leave (1) / the finished system can be checked against what the practice agreed (1) 2. Any three from: overview/introduction to the system (1) / hardware/software requirements and installation instructions (1) / user guide with step-by-step instructions/screenshots, e.g. how to book or cancel an appointment (1) / troubleshooting section/error messages explained (1) / FAQ section (1) / training materials/tutorials (1) / backup/maintenance procedures (1) 3. Any three from: system specification/user requirements (1) / data models, DFDs, ER diagrams, data dictionary (1) / input/output design (1) / report specifications (1) / query designs (1) / program code listings/annotated code (1) / test plans with data used and results (1) / detailed hardware and software configuration (1)